

Notes

Purpose: This No-operation filter receives ELR messages from the *ECLIR - ELR 2.5.1 Message Parsing* route.

Destinations

Name	Page
js Create XML to query NBS_SRTE..... JavaScript (v.1)	456
js Extract NBS_SRTE Errors..... JavaScript (v.1)	467
js Set Message Error properties..... JavaScript (v.1)	411
NBS Definition Validation..... HL7 Acknowledgement Generation	481
NIST Definition Validation..... HL7 Acknowledgement Generation	482

**db Add Field Descriptions**

Filter

Database Message Extraction

On ECLIR - ELR 2.5.1 Message Validation route

Configuration Properties		
Maximum Concurrency	1	
Database		
Host		
Port		
Database Name		
Username		
Password		
Additional Connection Properties	Property Name	Property Value
Configuration File		
Message Body Column		
Message Type	XML	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	
Configuration File:		
<pre><statement name="errorRecord" ide:type="executeSql" iterate="errorRecord" ide:summary="Adds the field description to each errorRecord node from the prior validations." ide:allowCompoundStatements="true"></pre>		

Configuration File:

```

<sql>SELECT `/message/errorRecord/VALIDATION_TYPE` AS VALIDATION_TYPE
, `/message/errorRecord/ELR_MESSAGE_ID` AS ELR_MESSAGE_ID
, `/message/errorRecord/ERROR_CATEGORY` AS ERROR_CATEGORY
, `/message/errorRecord/PROCESSING_STEP` AS PROCESSING_STEP
, `/message/errorRecord/FIELD` AS FIELD
, COALESCE(
(
SELECT TOP(1) COLUMN_DESCRIPTION
FROM ELR_HL7_FIELD_DESCRIPTIONS
WHERE HL7_FIELD_NOTATION = `/message/errorRecord/FIELD`
OR HL7_FIELD_NOTATION = LEFT(`/message/errorRecord/FIELD`, LEN(`/message/errorRecord/
FIELD`) - charindex('.', reverse(`/message/errorRecord/FIELD`)))
OR HL7_FIELD_NOTATION = LEFT(`/message/errorRecord/FIELD`, LEN(`/message/errorRecord/
FIELD`) - (charindex('.', reverse(`/message/errorRecord/FIELD`), charindex('.', reverse(`/mes
sage/errorRecord/FIELD`))+1)))
ORDER BY HL7_FIELD_NOTATION DESC
), NULL) AS FIELD_DESCRIPTION
, `/message/errorRecord/FIELD_PATH` AS FIELD_PATH
, `/message/errorRecord/FIELD_VALUE` AS FIELD_VALUE
, `/message/errorRecord/ERROR_MSG` AS ERROR_MSG
, `/message/errorRecord/ERROR_TYPE` AS ERROR_TYPE
, `/message/errorRecord/ERROR_TYPE_DETAIL` AS ERROR_TYPE_DETAIL</sql>
</statement>
</config>

```

Destinations

Name	Page
 no-op ELR Out to Errors Route..... No-operation	410



no-op REH to Sink

No-operation

On ECLIR - ELR 2.5.1 Message Validation route

Filter

Notes

Purpose: This No-operation filter sends messages from the *ECLIR - ELR 2.5.1 Message Validation Route Error Handler* to the *ECLIR - ELR Route Error Handler Sink* after the errors have been stored into the ELR_ERROR table in the ECLIR database.

Destinations

Name	Page
 ECLIR - ELR Route Error Handler Sink..... Sink	13



js Create XML to query NBS_SRTE

JavaScript (v.1)

Filter

On ECLIR - ELR 2.5.1 Message Validation route

Notes

Purpose: In this JavaScript filter:

The main JavaScript code traverses each segment and ordered group according to the message definition ECLIR_ELOR_ORU_2.5.1_NIST_validation.s3d for an ORU – Unsolicited Observation Message (Event R01). This message definition is used because it provides a less stringent structure than the ECLIR_ELOR_ORU_2.5.1_NBS_validation.s3d definition to allow for validation of repeating fields if present.

Data is parsed from the specific fields in the message that will be cross-referenced with a code set in a table in the NBS_SRTE database. This data, along with other relevant fields, is written to an XML flat file in order to easily iterate through elements to query the NBS_SRTE database in the *db NBS_SRTE Query Results* Database Lookup filter that follows.

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);
var xmlSRTE_Code_value_general = input[1];
var errList = '';
var dbInsertsXML = new XML(<message></message>);
var nodeCode_value_general = new XML(<Code_value_general></Code_value_general>);
var nodeV_Race_code = new XML(<V_Race_code></V_Race_code>);
var nodeLanguage_code = new XML(<Language_code></Language_code>);
var nodeV_State_county_code_value = new XML(<V_State_county_code_value></V_State_county_code_value>);
var nodeState_code = new XML(<State_code></State_code>);
var nodeLabtest_loinc = new XML(<Labtest_loinc></Labtest_loinc>);
var nodeLOINC_condition = new XML(<LOINC_condition></LOINC_condition>);
var nodeLOINC_code = new XML(<LOINC_code></LOINC_code>);
var nodeSnomed_condition = new XML(<Snomed_condition></Snomed_condition>);
var nodeSnomed_code = new XML(<Snomed_code></Snomed_code>);
var tempEle;

//
=====
// _____ INITIAL VARIABLES (end
)
//
=====

//
=====
// _____ FUNCTIONS (start
)
//
=====
```

Javascript

```

//FUNCTION: fnCreateXMLEntry
//Function to create an XML node for checking codes and text.
function fnCreateXMLEntry (pathHL7, xmlNodeName, codeSetName) {
    var dbColumnNode;
    var fieldData = next.getField(pathHL7);
    if (fieldData != null && fieldData != '') {
        tempEle = new XML(<query_record></query_record>);
        dbColumnNode = <code_set_nm>{codeSetName}</code_set_nm>;
        tempEle.appendChild(dbColumnNode);
        dbColumnNode = <code>{fieldData}</code>;
        tempEle.appendChild(dbColumnNode);
        dbColumnNode = <code_desc_txt>{fieldData}</code_desc_txt>;
        tempEle.appendChild(dbColumnNode);
        dbColumnNode = <code_short_desc_txt>{fieldData}</code_short_desc_txt>;
        tempEle.appendChild(dbColumnNode);
        dbColumnNode = <field_path>{pathHL7}</field_path>;
        tempEle.appendChild(dbColumnNode);
        xmlNodeName.appendChild(tempEle);
    }
}

//FUNCTION: fnCreateXMLLocalCheckEntry
//Function to create an XML node for checking LOINC codes and local codes.
function fnCreateXMLLocalCheckEntry (pathHL7, xmlNodeName) {
    var dbColumnNode;
    var fieldData = next.getField(pathHL7);
    if (fieldData != null && fieldData != '') {
        tempEle = new XML(<query_record></query_record>);
        dbColumnNode = <lab_id>{next.getField('MSH/SendingFacility/UniversalID')}</lab_id>;
        tempEle.appendChild(dbColumnNode);
        dbColumnNode = <lab_test_cd>{fieldData}</lab_test_cd>;
        tempEle.appendChild(dbColumnNode);
        dbColumnNode = <field_path>{pathHL7}</field_path>;
        tempEle.appendChild(dbColumnNode);
        xmlNodeName.appendChild(tempEle);
    }
}

//FUNCTION: fnCreateXMLLOINCEntry
//Function to create an XML node for checking LOINC codes in
// LOINC_condition and LOINC_code tables.
function fnCreateXMLLOINCEntry (pathHL7, xmlNodeName) {
    var dbColumnNode;
    var fieldData = next.getField(pathHL7);
    if (fieldData != null && fieldData != '') {
        tempEle = new XML(<query_record></query_record>);
        dbColumnNode = <loinc_cd>{fieldData}</loinc_cd>;
        tempEle.appendChild(dbColumnNode);
        dbColumnNode = <field_path>{pathHL7}</field_path>;
        tempEle.appendChild(dbColumnNode);
        xmlNodeName.appendChild(tempEle);
    }
}

```

Javascript

```

//FUNCTION: fnCreateXMLSNOMEDEntry
//Function to create an XML node for checking LOINC codes in
// SNOMED_condition and SNOMED_code tables.
function fnCreateXMLSNOMEDEntry (pathHL7, xmlNodeName) {
  var dbColumnNode;
  var fieldData = next.getField(pathHL7);
  if (fieldData != null && fieldData != '') {
    tempEle = new XML(<query_record></query_record>);
    dbColumnNode = <snomed_cd>{fieldData}</snomed_cd>;
    tempEle.appendChild(dbColumnNode);
    dbColumnNode = <field_path>{pathHL7}</field_path>;
    tempEle.appendChild(dbColumnNode);
    xmlNodeName.appendChild(tempEle);
  }
}

//
=====
//_____ FUNCTIONS (end
)
//
=====

//
=====
//_____ Build Output XML file (start
)
//
=====

//Create an XML output file to be used to query the NBS_SRTE tables to validate codes or text.

//MSH
fnCreateXMLEntry ('MSH/SendingFacility/UniversalIDType', nodeCode_value_general, 'EI_AUTH');

//----- PATIENT_RESULT group -----
for (i=0; i < next.getRepeatCount('PATIENT_RESULT'); i++) {
  var PID = 'PATIENT_RESULT['+i+']/PATIENT/PID';

  //----- PID segment -----
  //PatientIdentifierList
  for (j=0; j < next.getRepeatCount(PID + '/PatientIdentifierList'); j++) {
    fnCreateXMLEntry (PID + '/PatientIdentifierList['+j+']/AssigningAuthority/UniversalIDType',
nodeCode_value_general, 'EI_TYPE');
    if ((next.getField(PID + '/PatientIdentifierList['+j+']/IdentifierTypeCode') != null) &&
(next.getField(PID + '/PatientIdentifierList['+j+']/IdentifierTypeCode') != '')) {
      fnCreateXMLEntry (PID + '/PatientIdentifierList['+j+']/IdentifierTypeCode', nodeCode_value
_general, 'EI_TYPE');
    }
    fnCreateXMLEntry (PID + '/PatientIdentifierList['+j+']/AssigningFacility/UniversalIDType',
nodeCode_value_general, 'EI_TYPE');
  }
  //PatientIdentifierList (end)
}

```

Javascript

```

//PatientName - ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (PID + '/PatientName[0]/Suffix', nodeCode_value_general, 'P_NM_SFX');
fnCreateXMLEntry (PID + '/PatientName[0]/Prefix', nodeCode_value_general, 'P_NM_PFX');
fnCreateXMLEntry (PID + '/PatientName[0]/Degree_DEPRECATED', nodeCode_value_general, 'P_NM_DEG');
fnCreateXMLEntry (PID + '/PatientName[0]/NameTypeCode', nodeCode_value_general, 'P_NM_USE');
//PatientName (end)

fnCreateXMLEntry (PID + '/AdministrativeSex', nodeCode_value_general, 'SEX');

//Race
for (j=0; j < next.getRepeatCount(PID + '/Race'); j++) {
  fnCreateXMLEntry (PID + '/Race['+j+']/Identifier', nodeV_Race_code, 'P_RACE');
}
//Race (end)

//PatientAddress - ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (PID + '/PatientAddress[0]/StateOrProvince', nodeState_code, 'STATE_CCD');
fnCreateXMLEntry (PID + '/PatientAddress[0]/Country', nodeCode_value_general, 'PSL_CNTRY');
fnCreateXMLEntry (PID + '/PatientAddress[0]/AddressType', nodeCode_value_general, 'EL_TYPE_PST_PAT');
fnCreateXMLEntry (PID + '/PatientAddress[0]/CountyParishCode', nodeV_State_county_code_value, 'COUNTY_CCD');
//PatientAddress (end)

//ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (PID + '/HomePhoneNumber[0]/TelecommunicationUseCode', nodeCode_value_general, 'EL_USE');
fnCreateXMLEntry (PID + '/HomePhoneNumber[0]/TelecommunicationEquipmentType', nodeCode_value_general, 'EI_TYPE');
fnCreateXMLEntry (PID + '/BusinessPhoneNumber[0]/TelecommunicationUseCode', nodeCode_value_general, 'EL_USE');
fnCreateXMLEntry (PID + '/BusinessPhoneNumber[0]/TelecommunicationEquipmentType', nodeCode_value_general, 'EI_TYPE');

//ODS Comment: Not treated as repeatable.
fnCreateXMLEntry (PID + '/PrimaryLanguage[0]/Identifier', nodeLanguage_code, 'P_LANG');

fnCreateXMLEntry (PID + '/MaritalStatus/Identifier', nodeCode_value_general, 'P_MARITAL');
fnCreateXMLEntry (PID + '/PatientAccountNumber/IdentifierTypeCode', nodeCode_value_general, 'EI_TYPE');
fnCreateXMLEntry (PID + '/DriversLicenseNumberPatient/IssuingStateProvinceCountry', nodeCode_value_general, 'EI_AUTH');

//ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (PID + '/MothersIdentifier[0]/IdentifierTypeCode', nodeCode_value_general, 'EI_TYPE');

fnCreateXMLEntry (PID + '/EthnicGroup/Identifier', nodeCode_value_general, 'P_ETHN');
fnCreateXMLEntry (PID + '/BirthPlace', nodeCode_value_general, 'PSL_CNTRY');
fnCreateXMLEntry (PID + '/MultipleBirthIndicator', nodeCode_value_general, 'YNU');
fnCreateXMLEntry (PID + '/PatientDeathIndicator', nodeCode_value_general, 'YNU');
//----- PID segment (end) -----

//----- NK1 segment -----
for (j=0; j < next.getRepeatCount('PATIENT_RESULT['+i+']/PATIENT/NK1'); j++) {

```

Javascript

```

var NK1 = 'PATIENT_RESULT['+i+']/PATIENT/NK1['+j+']';
//ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (NK1 + '/Name[0]/Suffix', nodeCode_value_general, 'P_NM_SFX');
fnCreateXMLEntry (NK1 + '/Name[0]/Prefix', nodeCode_value_general, 'P_NM_PFX');
fnCreateXMLEntry (NK1 + '/Name[0]/Degree_DEPRECATED', nodeCode_value_general, 'P_NM_DEG');
fnCreateXMLEntry (NK1 + '/Name[0]/NameTypeCode', nodeCode_value_general, 'P_NM_USE');

fnCreateXMLEntry (NK1 + '/Relationship/Identifier', nodeCode_value_general, 'RL_TYPE');

//ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (NK1 + '/Address[0]/StateOrProvince', nodeState_code, 'STATE_CCD');
fnCreateXMLEntry (NK1 + '/Address[0]/Country', nodeCode_value_general, 'PSL_CNTRY');
fnCreateXMLEntry (NK1 + '/Address[0]/AddressType', nodeCode_value_general, 'EL_TYPE_PST
PAT');
fnCreateXMLEntry (NK1 + '/Address[0]/CountyParishCode', nodeV_State_county_code_value,
'COUNTY_CCD');

//ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (NK1 + '/PhoneNumber[0]/TelecommunicationUseCode', nodeCode_value_general,
'EL_USE');
fnCreateXMLEntry (NK1 + '/PhoneNumber[0]/TelecommunicationEquipmentType', nodeCode_value_gen
eral, 'EI_TYPE');
}
//----- NK1 segment (end) -----

//----- ORDER_OBSERVATION group -----
for (j=0; j < next.getRepeatCount('PATIENT_RESULT['+i+']/ORDER_OBSERVATION'); j++) {
  var ORC = 'PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/ORC';
  var OBR = 'PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/OBR';

  //----- ORC segment -----
  //ODS Comment: Presently NBS stores only one value
  fnCreateXMLEntry (ORC + '/OrderingFacilityName[0]/OrganizationNameTypeCode', nodeCode_value
_general, 'O_NM_USE');

  //ODS Comment: Presently NBS stores only one value
  fnCreateXMLEntry (ORC + '/OrderingFacilityAddress[0]/StateOrProvince', nodeState_code, 'STATE
_CCD');
  fnCreateXMLEntry (ORC + '/OrderingFacilityAddress[0]/Country', nodeCode_value_general, 'PSL
_CNTRY');
  fnCreateXMLEntry (ORC + '/OrderingFacilityAddress[0]/CountyParishCode', nodeV_State_county
_code_value, 'COUNTY_CCD');

  //ODS Comment: Presently NBS stores only one value
  fnCreateXMLEntry (ORC + '/OrderingProviderAddress[0]/StateOrProvince', nodeState_code, 'STATE
_CCD');
  fnCreateXMLEntry (ORC + '/OrderingProviderAddress[0]/Country', nodeCode_value_general, 'PSL
_CNTRY');
  fnCreateXMLEntry (ORC + '/OrderingProviderAddress[0]/CountyParishCode', nodeV_State_county
_code_value, 'COUNTY_CCD');
  //----- ORC segment (end) -----

  //----- OBR segment -----
  //-----OBR-4-----

```

Javascript

```

//Translate any local codes to a LOINC code and check the resulting LOINC code against other
tables.
if (next.getField(OBR + '/UniversalServiceIdentifier/NameOfCodingSystem') == 'L') {
  fnCreateXMLLocalCheckEntry (OBR + '/UniversalServiceIdentifier/Identifier', nodeLabtest
_loinc);
}
if (next.getField(OBR + '/UniversalServiceIdentifier/NameOfAlternateCodingSystem') == 'L') {
  fnCreateXMLLocalCheckEntry (OBR + '/UniversalServiceIdentifier/AlternateIdentifier', node
Labtest_loinc);
}

//Check LOINC codes against LOINC_condition and LOINC_code tables.
if (next.getField(OBR + '/UniversalServiceIdentifier/NameOfCodingSystem') == 'LN') {
  var tempPath = OBR + '/UniversalServiceIdentifier/Identifier';
  fnCreateXMLLOINCEntry (tempPath, nodeLOINC_code);
  fnCreateXMLLOINCEntry (tempPath, nodeLOINC_condition);
}
if (next.getField(OBR + '/UniversalServiceIdentifier/NameOfAlternateCodingSystem') == 'LN')
{
  var tempPath = OBR + '/UniversalServiceIdentifier/AlternateIdentifier';
  fnCreateXMLLOINCEntry (tempPath, nodeLOINC_code);
  fnCreateXMLLOINCEntry (tempPath, nodeLOINC_condition);
}
//-----OBR-4 (end)-----

//ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (OBR + '/CollectorIdentifier[0]/Suffix', nodeCode_value_general, 'P_NM
_SFX');
fnCreateXMLEntry (OBR + '/CollectorIdentifier[0]/Prefix', nodeCode_value_general, 'P_NM
_PFX');
fnCreateXMLEntry (OBR + '/CollectorIdentifier[0]/Degree_DEPRECATED', nodeCode_value_general,
'P_NM_DEG');

fnCreateXMLEntry (OBR + '/SpecimenActionCode', nodeCode_value_general, 'M_HANDLING');
fnCreateXMLEntry (OBR + '/DangerCode/Identifier', nodeCode_value_general, 'M_RISK');

//ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (OBR + '/SpecimenSource[0]/SpecimenSourceNameOrCode/Identifier', nodeCode
_value_general, 'SPECMN_SRC');

//ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (OBR + '/OrderingProvider[0]/Suffix', nodeCode_value_general, 'P_NM_SFX');
fnCreateXMLEntry (OBR + '/OrderingProvider[0]/Prefix', nodeCode_value_general, 'P_NM_PFX');
fnCreateXMLEntry (OBR + '/OrderingProvider[0]/Degree_DEPRECATED', nodeCode_value_general, 'P
_NM_DEG');

fnCreateXMLEntry (OBR + '/ResultStatus', nodeCode_value_general, 'ACT_OBJ_ST');

for (k=0; k < next.getRepeatCount(OBR + '/ResultCopiesTo'); k++) {
  fnCreateXMLEntry (OBR + '/ResultCopiesTo['+k+']/Suffix', nodeCode_value_general, 'P_NM
_SFX');
  fnCreateXMLEntry (OBR + '/ResultCopiesTo['+k+']/Prefix', nodeCode_value_general, 'P_NM
_PFX');
  fnCreateXMLEntry (OBR + '/ResultCopiesTo['+k+']/Degree_DEPRECATED', nodeCode_value_general,
'P_NM_DEG');
}

for (k=0; k < next.getRepeatCount(OBR + '/ReasonForStudy'); k++) {

```

Javascript

```

    fnCreateXMLEntry (OBR + '/ReasonForStudy['+k+']/Identifier', nodeCode_value_general, 'OBS
_REASON');
}

    fnCreateXMLEntry (OBR + '/PrincipalResultInterpreter/Name/Suffix', nodeCode_value_general, 'P
_NM_SFX');
    fnCreateXMLEntry (OBR + '/PrincipalResultInterpreter/Name/Prefix', nodeCode_value_general, 'P
_NM_PFX');
    fnCreateXMLEntry (OBR + '/PrincipalResultInterpreter/Name/Degree_DEPRECATED', nodeCode_value
_general, 'P_NM_DEG');
    fnCreateXMLEntry (OBR + '/PrincipalResultInterpreter/Name/AssigningAuthorityNamespaceID',
nodeCode_value_general, 'EI_AUTH');
//----- OBR segment (end) -----

//----- OBSERVATION group -----

for (k=0; k < next.getRepeatCount('PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/OBSERVA
TION'); k++) {
    var OBX = 'PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/OBSERVATION['+k+']/OBX';
    //----- OBX segment -----

    //-----OBX-3-----
    //Translate any local codes to a LOINC code and check the resulting LOINC code against other
tables.

    if (next.getField(OBX + '/ObservationIdentifier/NameOfCodingSystem') == 'L') {
        fnCreateXMLLocalCheckEntry (OBX + '/ObservationIdentifier/Identifier', nodeLabtest_loinc);
    }
    if (next.getField(OBX + '/ObservationIdentifier/NameOfAlternateCodingSystem') == 'L') {
        fnCreateXMLLocalCheckEntry (OBX + '/ObservationIdentifier/AlternateIdentifier', nodeLabtest
_loinc);
    }

    //Check LOINC codes against LOINC_condition and LOINC_code tables.
    if (next.getField(OBX + '/ObservationIdentifier/NameOfCodingSystem') == 'LN') {
        var tempPath = OBX + '/ObservationIdentifier/Identifier';
        fnCreateXMLLOINCEntry (tempPath, nodeLOINC_code);
        fnCreateXMLLOINCEntry (tempPath, nodeLOINC_condition);
    }
    if (next.getField(OBX + '/ObservationIdentifier/NameOfAlternateCodingSystem') == 'LN') {
        var tempPath = OBX + '/ObservationIdentifier/AlternateIdentifier';
        fnCreateXMLLOINCEntry (tempPath, nodeLOINC_code);
        fnCreateXMLLOINCEntry (tempPath, nodeLOINC_condition);
    }
}
//-----OBX-3 (end)-----

//-----OBX-5-----
//Handle OBX-5 variations based on the data type in OBX-2
var OBX5DataType = next.getField(OBX + '/ValueType');
if ((OBX5DataType == 'CE') || (OBX5DataType == 'CWE')) {
    var OBX5CodingSys = next.getField(OBX + '/ObservationValue/NameOfCodingSystem');
    if ((OBX5CodingSys == 'SN') || (OBX5CodingSys == 'SNM')) {
        fnCreateXMLSNOMEDEntry (OBX + '/ObservationValue/Identifier', nodeSnomed_code);
        fnCreateXMLSNOMEDEntry (OBR + '/ObservationValue/Identifier', nodeSnomed_condition);
    }
}

```

Javascript

```

var OBX5AltCodingSys = next.getField(OBX + '/ObservationValue/NameOfAlternateCodingSystem
');
if ((OBX5AltCodingSys == 'SN') || (OBX5AltCodingSys == 'SNM')) {
    fnCreateXMLSNOMEDEntry (OBX + '/ObservationValue/AlternateIdentifier', nodeSnomed_code);
    fnCreateXMLSNOMEDEntry (OBX + '/ObservationValue/AlternateIdentifier', nodeSnomed_con
dition);
}
}
if (OBX5DataType == 'SN') {
    fnCreateXMLEntry (OBX + '/ObservationValue/Comparator', nodeCode_value_general, 'OV_NUM
_CMP');
    fnCreateXMLEntry (OBX + '/ObservationValue/SeparatorSuffix', nodeCode_value_general, 'OV
_NUM_SEP');
}
//-----OBX-5 (end)-----

fnCreateXMLEntry (OBX + '/Units/Identifier', nodeCode_value_general, 'UNIT_ISO');

//ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (OBX + '/AbnormalFlags[0]/Identifier', nodeCode_value_general, 'OBS_INTRP
');

fnCreateXMLEntry (OBX + '/ObservationResultStatus', nodeCode_value_general, 'ACT_OBJ_ST');
fnCreateXMLEntry (OBX + '/ProducersID/NameOfCodingSystem', nodeCode_value_general, 'EI
_AUTH');

//ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (OBX + '/ObservationMethod[0]/Identifier', nodeCode_value_general, 'OBS
_METH');

fnCreateXMLEntry (OBX + '/PerformingOrganizationName/IDNumber_DEPRECATED', nodeCode_value
_general, 'EI_AUTH');

fnCreateXMLEntry (OBX + '/PerformingOrganizationAddress/StateOrProvince', nodeState_code,
'STATE_CCD');
fnCreateXMLEntry (OBX + '/PerformingOrganizationAddress/Country', nodeCode_value_general,
'PSL_CNTRY');
fnCreateXMLEntry (OBX + '/PerformingOrganizationAddress/CountyParishCode', nodeV_State
_county_code_value, 'COUNTY_CCD');
//----- OBX segment (end) -----

}
//----- OBSERVATION group (end) -----

//----- SPECIMEN group -----
for (k=0; k < next.getRepeatCount('PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/SPECIMEN');
k++) {
    var SPM = 'PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/SPECIMEN['+k+']/SPM';

    //----- SPM segment -----
    fnCreateXMLEntry (SPM + '/SpecimenType/Identifier', nodeCode_value_general, 'SPECMN_SRC');
    fnCreateXMLEntry (SPM + '/SpecimenSourceSite/Identifier', nodeCode_value_general, 'ANATOMIC
_SITE');
    //----- SPM segment (end) -----

```

Javascript

```

    for (l=0; l < next.getRepeatCount('PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/SPECIMEN
['+k+']/OBX'); l++) {
        var SPM_OBX = 'PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/SPECIMEN['+k+']/OBX['+l+']';

        //----- OBX segments -----

        //-----OBX-3-----
        //Translate any local codes to a LOINC code and check the resulting LOINC code against
        other tables.
        if (next.getField(SPM_OBX + '/ObservationIdentifier/NameOfCodingSystem') == 'L') {
            fnCreateXMLLocalCheckEntry (SPM_OBX + '/ObservationIdentifier/Identifier', nodeLabtest
_loinc);
        }
        if (next.getField(SPM_OBX + '/ObservationIdentifier/NameOfAlternateCodingSystem') == 'L')
        {
            fnCreateXMLLocalCheckEntry (SPM_OBX + '/ObservationIdentifier/AlternateIdentifier', node
Labtest_loinc);
        }

        //Check LOINC codes against LOINC_condition and LOINC_code tables.
        if (next.getField(SPM_OBX + '/ObservationIdentifier/NameOfCodingSystem') == 'LN') {
            var tempPath = SPM_OBX + '/ObservationIdentifier/Identifier';
            fnCreateXMLLOINCEntry (tempPath, nodeLOINC_code);
            fnCreateXMLLOINCEntry (tempPath, nodeLOINC_condition);
        }
        if (next.getField(SPM_OBX + '/ObservationIdentifier/NameOfAlternateCodingSystem') == 'LN')
        {
            var tempPath = SPM_OBX + '/ObservationIdentifier/AlternateIdentifier';
            fnCreateXMLLOINCEntry (tempPath, nodeLOINC_code);
            fnCreateXMLLOINCEntry (tempPath, nodeLOINC_condition);
        }
        //-----OBX-3 (end)-----

        //-----OBX-5-----
        //Handle OBX-5 variations based on the data type in OBX-2
        var OBX5DataType = next.getField(SPM_OBX + '/ValueType');
        if ((OBX5DataType == 'CE') || (OBX5DataType == 'CWE')) {
            var OBX5CodingSys = next.getField(SPM_OBX + '/ObservationValue/NameOfCodingSystem');
            if ((OBX5CodingSys == 'SN') || (OBX5CodingSys == 'SNM')) {
                fnCreateXMLSNOMEDEntry (SPM_OBX + '/ObservationValue/Identifier', nodeSnomed_code);
                fnCreateXMLSNOMEDEntry (SPM_OBX + '/ObservationValue/Identifier', nodeSnomed_condition);
            }
            var OBX5AltCodingSys = next.getField(SPM_OBX + '/ObservationValue/NameOfAlternateCoding
System');
            if ((OBX5AltCodingSys == 'SN') || (OBX5AltCodingSys == 'SNM')) {
                fnCreateXMLSNOMEDEntry (SPM_OBX + '/ObservationValue/AlternateIdentifier', nodeSnomed
_code);
                fnCreateXMLSNOMEDEntry (SPM_OBX + '/ObservationValue/AlternateIdentifier', nodeSnomed_con
dition);
            }
        }
        if (OBX5DataType == 'SN') {
            fnCreateXMLEntry (SPM_OBX + '/ObservationValue/Comparator', nodeCode_value_general, 'OV
_NUM_CMP');
            fnCreateXMLEntry (SPM_OBX + '/ObservationValue/SeparatorSuffix', nodeCode_value_general,
'OV_NUM_SEP');
        }
    }

```

Javascript

```

    }
    //-----OBX-5 (end)-----

    fnCreateXMLEntry (SPM_OBX + '/Units/Identifier', nodeCode_value_general, 'UNIT_ISO');

    //ODS Comment: Presently NBS stores only one value
    fnCreateXMLEntry (SPM_OBX + '/AbnormalFlags[0]/Identifier', nodeCode_value_general, 'OBS
_INTRP');

    fnCreateXMLEntry (SPM_OBX + '/ObservationResultStatus', nodeCode_value_general, 'ACT_OBJ
_ST');
    fnCreateXMLEntry (SPM_OBX + '/ProducersID/NameOfCodingSystem', nodeCode_value_general, 'EI
_AUTH');
    //ODS Comment: Presently NBS stores only one value
    fnCreateXMLEntry (SPM_OBX + '/ObservationMethod[0]/Identifier', nodeCode_value_general,
'OBS_METH');
    fnCreateXMLEntry (SPM_OBX + '/PerformingOrganizationName/IDNumber_DEPRECATED', nodeCode
_value_general, 'EI_AUTH');

    fnCreateXMLEntry (SPM_OBX + '/PerformingOrganizationAddress/StateOrProvince', nodeState
_code, 'STATE_CCD');
    fnCreateXMLEntry (SPM_OBX + '/PerformingOrganizationAddress/Country', nodeCode_value_gen
eral, 'PSL_CNTRY');
    fnCreateXMLEntry (SPM_OBX + '/PerformingOrganizationAddress/CountyParishCode', nodeV_State
_county_code_value, 'COUNTY_CCD');
    //----- OBX segments (end) -----

    }

    } // -----(END) for (k=0; k < next.getRepeatCount('PATIENT_RESULT['+i+']/ORDER_OBSERVATION
['+j+']/SPECIMEN'); k++) (END)
    //----- SPECIMEN group (end) -----

    } // -----(END) for (j=0; j < next.getRepeatCount('PATIENT_RESULT['+i+']/ORDER_OBSERVA
TION'); j++) (END)
    //--- ORDER_OBSERVATION group (end) ---

    } // -----(END) for (i=0; i < next.getRepeatCount('PATIENT_RESULT'); i++) (END)
    //---- PATIENT_RESULT group (end) ----

    //
    =====
    //_____ Build Output XML file (end
    )
    //
    =====

dbInsertsXML.appendChild(nodeCode_value_general);
dbInsertsXML.appendChild(nodeV_Race_code);
dbInsertsXML.appendChild(nodeLanguage_code);
dbInsertsXML.appendChild(nodeV_State_county_code_value);

```

Javascript

```
dbInsertsXML.appendChild(nodeState_code);
dbInsertsXML.appendChild(nodeLabtest_loinc);
dbInsertsXML.appendChild(nodeLOINC_condition);
dbInsertsXML.appendChild(nodeLOINC_code);
dbInsertsXML.appendChild(nodeSnomed_condition);
dbInsertsXML.appendChild(nodeSnomed_code);
```

```
next.text = dbInsertsXML;
```

Destinations

Name	Page
 db NBS_SRTE Query Results..... Database Message Extraction	451

**js Extract NBS_SRTE Errors**

Filter

JavaScript (v.1)

On ECLIR - ELR 2.5.1 Message Validation route

Notes

Purpose: In this JavaScript filter, the code traverses each segment and ordered group according to the message definition ECLIR_ELORU_2.5.1_NIST_validation.s3d for an ORU – Unsolicited Observation Message (Event R01). This message definition is used because it provides a less stringent structure than the ECLIR_ELORU_2.5.1_NBS_validation.s3d definition to allow for validation of repeating fields if present.

A message collector is used to input 2 messages into the filter.

The first message is the output XML file received from the *db NBS_SRTE Query Results* Database Lookup filter.

The second message is the original HL7 v2 ELR message.

Specific fields in the message that should have a valid value or code in a code set within a corresponding table in the NBS_SRTE database are searched for in the XML file received from the *db NBS_SRTE Query Results* Database Lookup filter. If an element does not exist for the specific field, this data, along with other relevant fields, is written to an XML flat file that follows the ECLIR - Error.xsd format for use to write these errors to the ECLIR database in the *ECLIR - ELR Errors* route.

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);
var ELRMessage = input[1];
var dbInsertsXML = new XML(<message></message>);
var tempEle;

//
=====
```

Javascript

```

// _____ INITIAL VARIABLES (end
)
//
=====

//
=====
// _____ FUNCTIONS (start
)
//
=====

//FUNCTION: fnCreateErrorMsg
//Function to create the exact error message to be written to the ELR_ERROR table.
function fnCreateErrorMsg (pathHL7, tblNBS_SRTE, codeSetName, fieldNotation, invalidType) {
  var errorMsg = '';
  switch (invalidType) {
    case 'Invalid_code':
      errorMsg += 'The code "' + ELRMessage.getField(pathHL7) + '" in field ' + fieldNotation +
        ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') was not found in the NBS_SRTE table "'
        + tblNBS_SRTE + '" for code set name "' + codeSetName + '".';
      break;
    case 'Invalid_LOINC_code':
    case 'Invalid_SNOMED_code':
      errorMsg += 'The code "' + ELRMessage.getField(pathHL7) + '" in field ' + fieldNotation +
        ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') was not found in the NBS_SRTE table "'
        + tblNBS_SRTE + '".';
      break;
    case 'Invalid_code_from_text':
      errorMsg += 'A code equivalent to field value "' + ELRMessage.getField(pathHL7) + '" in
        field ' + fieldNotation + ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') was not found
        in NBS_SRTE table "' + tblNBS_SRTE + '".';
      break;
    case 'Invalid_local_code':
      errorMsg += 'The local code "' + ELRMessage.getField(pathHL7) + '" in field ' + field
        Notation + ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') could not be converted to a
        LOINC code from the NBS_SRTE table "' + tblNBS_SRTE + '".';
      break;
    case 'Condition_not_found_for_LOINC':
    case 'Condition_not_found_for_SNOMED':
      errorMsg += 'The ' + tblNBS_SRTE.substr(0, tblNBS_SRTE.indexOf('_')) + ' code "' + ELRM
        essage.getField(pathHL7) + '" in field ' + fieldNotation + ' (' + pathHL7.substr(pathHL7.last
        IndexOf('/')+1) + ') did not have a matching condition in the NBS_SRTE table "' + tblNBS_SRTE +
        '".';
      break;
    default:
      errorMsg += 'The code "' + ELRMessage.getField(pathHL7) + '" in field ' + fieldNotation +
        ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') was not found in the NBS_SRTE table "'
        + tblNBS_SRTE + '" for code set name "' + codeSetName + '".';
      break;
  }
  return errorMsg;
}

//FUNCTION: fnCheckIfRecordReturned

```

Javascript

```

//Function to check if a record was returned for the value in the field.
function fnCheckIfRecordReturned (requiredField, pathHL7, tblNBS_SRTE, codeSetName, fieldNotation, errorCategory, errorTypeDetail) {
  //If the field is required, check for returned record regardless of value or
  // if the field is not required, check for a returned record if valued.
  if (requiredField || (ELRMessage.getField(pathHL7) != null && ELRMessage.getField(pathHL7) != '')) {
    var errorMsg = fnCreateErrorMsg (pathHL7, tblNBS_SRTE, codeSetName, fieldNotation, errorTypeDetail);

    //Check to see if a code was found in the NBS_SRTE.
    // If there is no existing node, the code was not found and an error is recorded.
    if (tblNBS_SRTE == 'Labtest_loinc') {
      if (next.getField("/message/" + tblNBS_SRTE + "[field_path='" + pathHL7 + "']/LocalToLOINC") == null) {
        fnCreateErrorRecord (pathHL7, fieldNotation, errorCategory, errorMsg);
      }
      else {
        if (next.getField("/message/Labtest_loinc/CheckLOINC_code[field_path='" + pathHL7 + "']/loinc_cd") == null) {
          errorMsg = fnCreateErrorMsg (pathHL7, 'LOINC_code', codeSetName, fieldNotation, 'Invalid LOINC_code');
          fnCreateErrorRecord (pathHL7, fieldNotation, errorCategory, errorMsg);
        }
        if (next.getField("/message/Labtest_loinc/CheckLOINC_condition[field_path='" + pathHL7 + "']/loinc_cd") == null) {
          errorMsg = fnCreateErrorMsg (pathHL7, 'LOINC_condition', codeSetName, fieldNotation, 'Condition_not_found_for_LOINC');
          fnCreateErrorRecord (pathHL7, fieldNotation, errorCategory, errorMsg);
        }
      }
    }
    else {
      if (next.getField('/message/' + tblNBS_SRTE + '[field_path="' + pathHL7 + '"]/field_path') == null) {
        fnCreateErrorRecord (pathHL7, fieldNotation, errorCategory, errorMsg);
      }
    }
  }
}

//FUNCTION: fnCreateErrorRecord
//Function to create an error record in the output XML to be saved to ELR_ERROR table.
function fnCreateErrorRecord (pathHL7, fieldNotation, errorCategory, errorMsg) {
  tempEle = new XML(<errorRecord></errorRecord>);
  dbColumnNode = <VALIDATION_TYPE>{'NBS'}</VALIDATION_TYPE>;
  tempEle.appendChild(dbColumnNode);
  dbColumnNode = <ELR_MESSAGE_ID>{next.getProperty('ELR_MESSAGE_ID')}</ELR_MESSAGE_ID>;
  tempEle.appendChild(dbColumnNode);
  dbColumnNode = <ERROR_CATEGORY>{errorCategory}</ERROR_CATEGORY>;
  tempEle.appendChild(dbColumnNode);
  dbColumnNode = <PROCESSING_STEP>{'Validation: ECLIR - ELR Validation (Route):js Extract NBS_SRTE Errors (Filter)'}</PROCESSING_STEP>;
  tempEle.appendChild(dbColumnNode);
  dbColumnNode = <FIELD>{fieldNotation}</FIELD>;
  tempEle.appendChild(dbColumnNode);
  dbColumnNode = <FIELD_PATH>{pathHL7}</FIELD_PATH>;

```

Javascript

```

tempEle.appendChild(dbColumnName);
if (pathHL7 != null && pathHL7 != '') {
  if (ELRMessage.getField(pathHL7) != null) {
    dbColumnName = <FIELD_VALUE>{ELRMessage.getField(pathHL7)}</FIELD_VALUE>;
    tempEle.appendChild(dbColumnName);
  }
}
dbColumnName = <ERROR_MSG>{errorMsg}</ERROR_MSG>;
tempEle.appendChild(dbColumnName);
dbColumnName = <ERROR_TYPE>{'NBS_SRTE CONFORMANCE'}</ERROR_TYPE>;
tempEle.appendChild(dbColumnName);
dbColumnName = <ERROR_TYPE_DETAIL>{'Invalid value'}</ERROR_TYPE_DETAIL>;
tempEle.appendChild(dbColumnName);
dbInsertsXML.appendChild(tempEle);
}

//
=====
// _____ FUNCTIONS (end
)
//
=====

//
=====
// _____ Build Output XML file (start
)
//
=====
//Create an XML output file to be used to create error records in the ELR_ERROR table if codes
are not valid.

//----- MSH segment -----
fnCheckIfRecordReturned (true, 'MSH/SendingFacility/UniversalIDType', 'Code_value_general', 'EI
_AUTH', 'MSH-4.3', 'WARNING', 'Invalid_code');

//----- PATIENT_RESULT group -----
for (i=0; i < ELRMessage.getRepeatCount('PATIENT_RESULT'); i++) {
  var PID = 'PATIENT_RESULT['+i+']/PATIENT/PID';

  //----- PID segment -----
  //PatientIdentifierList
  for (j=0; j < ELRMessage.getRepeatCount(PID + '/PatientIdentifierList'); j++) {
    fnCheckIfRecordReturned (true, PID + '/PatientIdentifierList['+j+']/AssigningAuthority/Uni
versalIDType', 'Code_value_general', 'EI_TYPE', 'PID-3.4.3', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (true, PID + '/PatientIdentifierList['+j+']/IdentifierTypeCode',
'Code_value_general', 'EI_TYPE', 'PID-3.5', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (true, PID + '/PatientIdentifierList['+j+']/AssigningFacility/Uni
versalIDType', 'Code_value_general', 'EI_TYPE', 'PID-3.6.3', 'WARNING', 'Invalid_code');
  }
  //PatientIdentifierList (end)

  //PatientName - ODS Comment: Presently NBS stores only one value
  fnCheckIfRecordReturned (false, PID + '/PatientName[0]/Suffix', 'Code_value_general', 'P_NM
_SFX', 'PID-5.4', 'INFO', 'Invalid_code');

```

Javascript

```

fnCheckIfRecordReturned (false, PID + '/PatientName[0]/Prefix', 'Code_value_general', 'P_NM_PFX', 'PID-5.5', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/PatientName[0]/Degree_DEPRECATED', 'Code_value_general', 'P_NM_DEG', 'PID-5.6', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/PatientName[0]/NameTypeCode', 'Code_value_general', 'P_NM_USE', 'PID-5.7', 'INFO', 'Invalid_code');
//PatientName (end)

fnCheckIfRecordReturned (false, PID + '/AdministrativeSex', 'Code_value_general', 'SEX', 'PID-8', 'WARNING', 'Invalid_code');

//Race
for (j=0; j < ELRMessage.getRepeatCount(PID + '/Race'); j++) {
  fnCheckIfRecordReturned (false, PID + '/Race['+j+']/Identifier', 'V_Race_code', 'P_RACE', 'PID-10', 'INFO', 'Invalid_code');
}
//Race (end)

//PatientAddress - ODS Comment: Presently NBS stores only one value
fnCheckIfRecordReturned (false, PID + '/PatientAddress[0]/StateOrProvince', 'State_code', 'STATE_CCD', 'PID-11.4', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/PatientAddress[0]/Country', 'Code_value_general', 'PSL_CNTRY', 'PID-11.6', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/PatientAddress[0]/AddressType', 'Code_value_general', 'EL_TYPE_PST_PAT', 'PID-11.7', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/PatientAddress[0]/CountyParishCode', 'V_State_county_code_value', 'COUNTY_CCD', 'PID-11.9', 'INFO', 'Invalid_code');
//PatientAddress (end)

//ODS Comment: Presently NBS stores only one value
fnCheckIfRecordReturned (false, PID + '/HomePhoneNumber[0]/TelecommunicationUseCode', 'Code_value_general', 'EL_USE', 'PID-13.2', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/HomePhoneNumber[0]/TelecommunicationEquipmentType', 'Code_value_general', 'EI_TYPE', 'PID-13.3', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/BusinessPhoneNumber[0]/TelecommunicationUseCode', 'Code_value_general', 'EL_USE', 'PID-14.2', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/BusinessPhoneNumber[0]/TelecommunicationEquipmentType', 'Code_value_general', 'EI_TYPE', 'PID-14.3', 'INFO', 'Invalid_code');

//ODS Comment: Not treated as repeatable.
fnCheckIfRecordReturned (false, PID + '/PrimaryLanguage[0]/Identifier', 'Language_code', 'P_LANG', 'PID-15.1', 'INFO', 'Invalid_code');

fnCheckIfRecordReturned (false, PID + '/MaritalStatus/Identifier', 'Code_value_general', 'P_MARITAL', 'PID-16.1', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/PatientAccountNumber/IdentifierTypeCode', 'Code_value_general', 'EI_TYPE', 'PID-18.5', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/DriversLicenseNumberPatient/IssuingStateProvinceCountry', 'Code_value_general', 'EI_AUTH', 'PID-20.2', 'INFO', 'Invalid_code');

//ODS Comment: Presently NBS stores only one value
fnCheckIfRecordReturned (false, PID + '/MothersIdentifier[0]/IdentifierTypeCode', 'Code_value_general', 'EI_TYPE', 'PID-21.5', 'INFO', 'Invalid_code');

fnCheckIfRecordReturned (false, PID + '/EthnicGroup/Identifier', 'Code_value_general', 'P_ETHN', 'PID-22.1', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/BirthPlace', 'Code_value_general', 'PSL_CNTRY', 'PID-23', 'INFO', 'Invalid_code');

```

Javascript

```

fnCheckIfRecordReturned (false, PID + '/MultipleBirthIndicator', 'Code_value_general', 'YNU',
'PID-24', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/PatientDeathIndicator', 'Code_value_general', 'YNU',
'PID-30', 'INFO', 'Invalid_code');
//----- PID segment (end) -----

//----- NK1 segment -----
for (j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT['+i+']/PATIENT/NK1'); j++) {
    var NK1 = 'PATIENT_RESULT['+i+']/PATIENT/NK1['+j+']';

    //ODS Comment: Presently NBS stores only one value
    fnCheckIfRecordReturned (false, NK1 + '/Name[0]/Suffix', 'Code_value_general', 'P_NM_SFX',
'NK1-2.4', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, NK1 + '/Name[0]/Prefix', 'Code_value_general', 'P_NM_PFX',
'NK1-2.5', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, NK1 + '/Name[0]/Degree_DEPRECATED', 'Code_value_general', 'P
_NM_DEG', 'NK1-2.6', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, NK1 + '/Name[0]/NameTypeCode', 'Code_value_general', 'P_NM
_USE', 'NK1-2.7', 'INFO', 'Invalid_code');

    fnCheckIfRecordReturned (false, NK1 + '/Relationship/Identifier', 'Code_value_general', 'RL
_TYPEB', 'NK1-3', 'INFO', 'Invalid_code');

    //ODS Comment: Presently NBS stores only one value
    fnCheckIfRecordReturned (false, NK1 + '/Address[0]/StateOrProvince', 'State_code', 'STATE
_CCD', 'NK1-4.4', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, NK1 + '/Address[0]/Country', 'Code_value_general', 'PSL
_CNTRY', 'NK1-4.6', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, NK1 + '/Address[0]/AddressType', 'Code_value_general', 'EL
_TYPE PST PAT', 'NK1-4.7', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, NK1 + '/Address[0]/CountyParishCode', 'V_State_county_code
_value', 'COUNTY_CCD', 'NK1-4.9', 'INFO', 'Invalid_code');

    //ODS Comment: Presently NBS stores only one value
    fnCheckIfRecordReturned (false, NK1 + '/PhoneNumber[0]/TelecommunicationUseCode', 'Code_value
_general', 'EL_USE', 'NK1-5.2', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, NK1 + '/PhoneNumber[0]/TelecommunicationEquipmentType', 'Code
_value_general', 'EI_TYPEB', 'NK1-5.3', 'INFO', 'Invalid_code');
}
//----- NK1 segment (end) -----

//----- ORDER OBSERVATION group -----
for (j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT['+i+']/ORDER_OBSERVATION'); j++) {
    var ORC = 'PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/ORC';
    var OBR = 'PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/OBR';
    //----- ORC segment -----
    //ODS Comment: Presently NBS stores only one value
    fnCheckIfRecordReturned (false, ORC + '/OrderingFacilityName[0]/OrganizationNameTypeCode',
'Code_value_general', 'O_NM_USE', 'ORC-21.2', 'INFO', 'Invalid_code');

    //ODS Comment: Presently NBS stores only one value
    fnCheckIfRecordReturned (false, ORC + '/OrderingFacilityAddress[0]/StateOrProvince', 'State
_code', 'STATE_CCD', 'ORC-22.4', 'INFO', 'Invalid_code');

```

Javascript

```

    fnCheckIfRecordReturned (false, ORC + '/OrderingFacilityAddress[0]/Country', 'Code_value_general', 'PSL_CNTRY', 'ORC-22.6', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, ORC + '/OrderingFacilityAddress[0]/CountyParishCode', 'V_State_county_code_value', 'COUNTY_CCD', 'ORC-22.9', 'INFO', 'Invalid_code');

    //ODS Comment: Presently NBS stores only one value
    fnCheckIfRecordReturned (false, ORC + '/OrderingProviderAddress[0]/StateOrProvince', 'State_code', 'STATE_CCD', 'ORC-24.4', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, ORC + '/OrderingProviderAddress[0]/Country', 'Code_value_general', 'PSL_CNTRY', 'ORC-24.6', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, ORC + '/OrderingProviderAddress[0]/CountyParishCode', 'V_State_county_code_value', 'COUNTY_CCD', 'ORC-24.9', 'INFO', 'Invalid_code');
    //----- ORC segment (end) -----

    //----- OBR segment -----
    //-----OBR-4-----
    //Translate any local codes to a LOINC code and check the resulting LOINC code against other tables.
    if (ELRMessage.getField(OBR + '/UniversalServiceIdentifier/NameOfCodingSystem') == 'L') {
        fnCheckIfRecordReturned (true, OBR + '/UniversalServiceIdentifier/Identifier', 'Labtest_loinc', '', 'OBR-4.1', 'WARNING', 'Invalid_local_code');
    }
    if (ELRMessage.getField(OBR + '/UniversalServiceIdentifier/NameOfAlternateCodingSystem') == 'L') {
        fnCheckIfRecordReturned (false, OBR + '/UniversalServiceIdentifier/AlternateIdentifier', 'Labtest_loinc', '', 'OBR-4.4', 'WARNING', 'Invalid_local_code');
    }

    //Check LOINC codes against LOINC_condition and LOINC_code tables.
    if (ELRMessage.getField(OBR + '/UniversalServiceIdentifier/NameOfCodingSystem') == 'LN') {
        fnCheckIfRecordReturned (true, OBR + '/UniversalServiceIdentifier/Identifier', 'LOINC_code', '', 'OBR-4.1', 'WARNING', 'Invalid_LOINC_code');
        fnCheckIfRecordReturned (true, OBR + '/UniversalServiceIdentifier/Identifier', 'LOINC_condition', '', 'OBR-4.1', 'WARNING', 'Condition_not_found_for_LOINC');
    }
    if (ELRMessage.getField(OBR + '/UniversalServiceIdentifier/NameOfAlternateCodingSystem') == 'LN') {
        fnCheckIfRecordReturned (false, OBR + '/UniversalServiceIdentifier/AlternateIdentifier', 'LOINC_code', '', 'OBR-4.4', 'WARNING', 'Invalid_LOINC_code');
        fnCheckIfRecordReturned (false, OBR + '/UniversalServiceIdentifier/AlternateIdentifier', 'LOINC_condition', '', 'OBR-4.4', 'WARNING', 'Condition_not_found_for_LOINC');
    }
    //-----OBR-4 (end)-----

    //ODS Comment: Presently NBS stores only one value
    fnCheckIfRecordReturned (false, OBR + '/CollectorIdentifier[0]/Suffix', 'Code_value_general', 'P_NM_SFX', 'OBR-10.5', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, OBR + '/CollectorIdentifier[0]/Prefix', 'Code_value_general', 'P_NM_PFX', 'OBR-10.6', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, OBR + '/CollectorIdentifier[0]/Degree_DEPRECATED', 'Code_value_general', 'P_NM_DEG', 'OBR-10.7', 'INFO', 'Invalid_code');

    fnCheckIfRecordReturned (false, OBR + '/SpecimenActionCode', 'Code_value_general', 'M_HANDLING', 'OBR-11', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, OBR + '/DangerCode/Identifier', 'Code_value_general', 'M_RISK', 'OBR-12.1', 'INFO', 'Invalid_code');

```

Javascript

```

//ODS Comment: Presently NBS stores only one value
fnCheckIfRecordReturned (false, OBR + '/SpecimenSource[0]/SpecimenSourceNameOrCode/Identifier', 'Code_value_general', 'SPECMN_SRC', 'OBR-15.1.1', 'INFO', 'Invalid_code');

//ODS Comment: Presently NBS stores only one value
fnCheckIfRecordReturned (false, OBR + '/OrderingProvider[0]/Suffix', 'Code_value_general', 'P_NM_SFX', 'OBR-16.5', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, OBR + '/OrderingProvider[0]/Prefix', 'Code_value_general', 'P_NM_PFX', 'OBR-16.6', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, OBR + '/OrderingProvider[0]/Degree_DEPRECATED', 'Code_value_general', 'P_NM_DEG', 'OBR-16.7', 'INFO', 'Invalid_code');

fnCheckIfRecordReturned (true, OBR + '/ResultStatus', 'Code_value_general', 'ACT_OBJ_ST', 'OBR-25', 'WARNING', 'Invalid_code');

for (k=0; k < ELRMessage.getRepeatCount(OBR + '/ResultCopiesTo'); k++) {
  fnCheckIfRecordReturned (false, OBR + '/ResultCopiesTo['+k+']/Suffix', 'Code_value_general', 'P_NM_SFX', 'OBR-28.5', 'INFO', 'Invalid_code');
  fnCheckIfRecordReturned (false, OBR + '/ResultCopiesTo['+k+']/Prefix', 'Code_value_general', 'P_NM_PFX', 'OBR-28.6', 'INFO', 'Invalid_code');
  fnCheckIfRecordReturned (false, OBR + '/ResultCopiesTo['+k+']/Degree_DEPRECATED', 'Code_value_general', 'P_NM_DEG', 'OBR-28.7', 'INFO', 'Invalid_code');
}

for (k=0; k < ELRMessage.getRepeatCount(OBR + '/ReasonForStudy'); k++) {
  fnCheckIfRecordReturned (false, OBR + '/ReasonForStudy['+k+']/Identifier', 'Code_value_general', 'OBS_REASON', 'OBR-31.1', 'INFO', 'Invalid_code');
}

fnCheckIfRecordReturned (false, OBR + '/PrincipalResultInterpreter/Name/Suffix', 'Code_value_general', 'P_NM_SFX', 'OBR-32.1.5', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, OBR + '/PrincipalResultInterpreter/Name/Prefix', 'Code_value_general', 'P_NM_PFX', 'OBR-32.1.6', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, OBR + '/PrincipalResultInterpreter/Name/Degree_DEPRECATED', 'Code_value_general', 'P_NM_DEG', 'OBR-32.1.7', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, OBR + '/PrincipalResultInterpreter/Name/AssigningAuthorityNamespaceID', 'Code_value_general', 'EI_AUTH', 'OBR-32.1.9', 'INFO', 'Invalid_code');
//----- OBR segment (end) -----

//----- OBSERVATION group -----

for (k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/OBSERVATION'); k++) {
  var OBX = 'PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/OBSERVATION['+k+']/OBX';

  //----- OBX segment -----

  //-----OBX-3-----
  //Translate any local codes to a LOINC code and check the resulting LOINC code against other tables.
  if (ELRMessage.getField(OBX + '/ObservationIdentifier/NameOfCodingSystem') == 'L') {
    fnCheckIfRecordReturned (true, OBX + '/ObservationIdentifier/Identifier', 'Labtest_loinc', 'OBX-3.1', 'WARNING', 'requiredField');
  }
}

```

Javascript

```

    if (ELRMessage.getField(OBX + '/ObservationIdentifier/NameOfAlternateCodingSystem') == 'L')
    {
        fnCheckIfRecordReturned (false, OBX + '/ObservationIdentifier/AlternateIdentifier', 'Lab
test_loinc', '', 'OBX-3.4', 'WARNING', 'Invalid_local_code');
    }

    //Check LOINC codes against LOINC_condition and LOINC_code tables.
    if (ELRMessage.getField(OBX + '/ObservationIdentifier/NameOfCodingSystem') == 'LN') {
        fnCheckIfRecordReturned (true, OBX + '/ObservationIdentifier/Identifier', 'LOINC_code', '',
'OBX-3.1', 'WARNING', 'Invalid_LOINC_code');
        fnCheckIfRecordReturned (true, OBX + '/ObservationIdentifier/Identifier', 'LOINC_con
dition', '', 'OBX-3.1', 'WARNING', 'Condition_not_found_for_LOINC');
    }
    if (ELRMessage.getField(OBX + '/ObservationIdentifier/NameOfAlternateCodingSystem') == 'LN')
    {
        fnCheckIfRecordReturned (false, OBX + '/ObservationIdentifier/AlternateIdentifier', 'LOINC
_code', '', 'OBX-3.4', 'WARNING', 'Invalid_LOINC_code');
        fnCheckIfRecordReturned (false, OBX + '/ObservationIdentifier/AlternateIdentifier', 'LOINC
_condition', '', 'OBX-3.4', 'WARNING', 'Condition_not_found_for_LOINC');
    }
    //-----OBX-3 (end)-----

    if (ELRMessage.getField(OBX + '/ObservationResultStatus') != 'X') {
        //-----OBX-5-----
        //Handle OBX-5 variations based on the data type in OBX-2
        var OBX5DataType = ELRMessage.getField(OBX + '/ValueType');

        var requiredOBX5orOBX8 = false;
        if ((ELRMessage.getField(OBX + '/ObservationValue') == null || ELRMessage.getField(OBX + '/
ObservationValue') == '')
            && (ELRMessage.getField(OBX + '/AbnormalFlags[0]') == null || ELRMessage.getField(OBX +
'/AbnormalFlags[0]') == '')) {
            requiredOBX5orOBX8 = true;
        }
        else { requiredOBX5orOBX8 = false; }

        if ((OBX5DataType == 'CE') || (OBX5DataType == 'CWE')) {
            var OBX5CodingSys = ELRMessage.getField(OBX + '/ObservationValue/NameOfCodingSystem');
            if ((OBX5CodingSys == 'SN') || (OBX5CodingSys == 'SNM')) {
                fnCheckIfRecordReturned (true, OBX + '/ObservationValue/Identifier', 'Snomed_code', '',
'OBX-5.1', 'WARNING', 'Invalid_SNOMED_code');
                fnCheckIfRecordReturned (true, OBX + '/ObservationValue/Identifier', 'Snomed_condition',
'', 'OBX-5.1', 'WARNING', 'Condition_not_found_for_SNOMED');
            }
            var OBX5AltCodingSys = next.getField(OBX + '/ObservationValue/NameOfAlternateCodingSystem
');
            if ((OBX5AltCodingSys == 'SN') || (OBX5AltCodingSys == 'SNM')) {
                fnCreateXMLSNOMEDEntry (false, OBX + '/ObservationValue/AlternateIdentifier', 'Snomed
_code', '', 'OBX-5.4', 'WARNING', 'Invalid_SNOMED_code');
                fnCreateXMLSNOMEDEntry (false, OBX + '/ObservationValue/AlternateIdentifier', 'Snomed_con
dition', '', 'OBX-5.4', 'WARNING', 'Condition_not_found_for_SNOMED');
            }
        }
        if (OBX5DataType == 'SN') {
            fnCheckIfRecordReturned (false, OBX + '/ObservationValue/Comparator', 'Code_value_gen
eral', 'OV_NUM_CMP', 'OBX-5.1', 'INFO', 'Invalid_code');
            fnCheckIfRecordReturned (false, OBX + '/ObservationValue/SeparatorSuffix', 'Code_value_gen
eral', 'OV_NUM_SEP', 'OBX-5.3', 'INFO', 'Invalid_code');

```

Javascript

```

    }
    //-----OBX-5 (end)-----

    //OBX-6 is required if OBX-2 is 'SN' or 'NM' and OBX-11 is not 'X'.
    fnCheckIfRecordReturned ((OBX5DataType == 'SN' || OBX5DataType == 'NM'), OBX + '/Units/Identifier', 'Code_value_general', 'UNIT_ISO', 'OBX-6.1', 'WARNING', 'Invalid_code');

    //ODS Comment: Presently NBS stores only one value
    fnCheckIfRecordReturned (requiredOBX5orOBX8, OBX + '/AbnormalFlags[0]/Identifier', 'Code_value_general', 'OBS_INTRP', 'OBX-8', 'INFO', 'Invalid_code');

    fnCheckIfRecordReturned (false, OBX + '/ObservationResultStatus', 'Code_value_general', 'ACT_OBJ_ST', 'OBX-11', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, OBX + '/ProducersID/NameOfCodingSystem', 'Code_value_general', 'EI_AUTH', 'OBX-15.3', 'INFO', 'Invalid_code');

    //ODS Comment: Presently NBS stores only one value
    fnCheckIfRecordReturned (false, OBX + '/ObservationMethod[0]/Identifier', 'Code_value_general', 'OBS_METH', 'OBX-17.1', 'INFO', 'Invalid_code');

    fnCheckIfRecordReturned (false, OBX + '/PerformingOrganizationName/IDNumber_DEPRECATED', 'Code_value_general', 'EI_AUTH', 'OBX-23.3', 'INFO', 'Invalid_code');

    fnCheckIfRecordReturned (false, OBX + '/PerformingOrganizationAddress/StateOrProvince', 'State_code', 'STATE_CCD', 'OBX-24.4', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, OBX + '/PerformingOrganizationAddress/Country', 'Code_value_general', 'PSL_CNTRY', 'OBX-24.6', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, OBX + '/PerformingOrganizationAddress/CountyParishCode', 'V_State_county_code_value', 'COUNTY_CCD', 'OBX-24.9', 'INFO', 'Invalid_code');
    //----- OBX segment (end) -----
    } // -----(END) if (ELRMessage.getField(OBX + '/ObservationResultStatus') != 'X') (END)

    } // -----(END) for (k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/OBSERVATION'); k++) (END)

    //----- OBSERVATION group (end) -----

    //----- SPECIMEN group -----
    for (k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/SPECIMEN'); k++) {
        var SPM = 'PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/SPECIMEN['+k+']';
        fnCheckIfRecordReturned (false, SPM + '/SPM/SpecimenType/Identifier', 'Code_value_general', 'SPECMN_SRC', 'SPM-4.1', 'INFO', 'Invalid_code');
        fnCheckIfRecordReturned (false, SPM + '/SPM/SpecimenSourceSite/Identifier', 'Code_value_general', 'ANATOMIC_SITE', 'SPM-8.1', 'INFO', 'Invalid_code');

        //----- OBX segments -----

        for (l=0; l < ELRMessage.getRepeatCount(SPM + '/OBX'); l++) {
            var SPM_OBX = SPM + '/OBX['+l+']';

            //-----OBX-3-----

```

Javascript

```

//Translate any local codes to a LOINC code and check the resulting LOINC code against
other tables.
if (ELRMessage.getField(SPM_OBX + '/ObservationIdentifier/NameOfCodingSystem') == 'L') {
    fnCheckIfRecordReturned (true, SPM_OBX + '/ObservationIdentifier/Identifier', 'Labtest
_loinc', '', 'OBX-3.1', 'WARNING', 'Invalid_local_code');
}
if (ELRMessage.getField(SPM_OBX + '/ObservationIdentifier/NameOfAlternateCodingSystem') ==
'L') {
    fnCheckIfRecordReturned (false, SPM_OBX + '/ObservationIdentifier/AlternateIdentifier',
'Labtest_loinc', '', 'OBX-3.4', 'WARNING', 'Invalid_local_code');
}

//Check LOINC codes against LOINC_condition and LOINC_code tables.
if (ELRMessage.getField(SPM_OBX + '/ObservationIdentifier/NameOfCodingSystem') == 'LN') {
    fnCheckIfRecordReturned (true, SPM_OBX + '/ObservationIdentifier/Identifier', 'LOINC
_code', '', 'OBX-3.1', 'WARNING', 'Invalid_LOINC_code');
    fnCheckIfRecordReturned (true, SPM_OBX + '/ObservationIdentifier/Identifier', 'LOINC_con
dition', '', 'OBX-3.1', 'WARNING', 'Condition_not_found_for_LOINC');
}
if (ELRMessage.getField(SPM_OBX + '/ObservationIdentifier/NameOfAlternateCodingSystem') ==
'LN') {
    fnCheckIfRecordReturned (false, SPM_OBX + '/ObservationIdentifier/AlternateIdentifier',
'LOINC_code', '', 'OBX-3.4', 'WARNING', 'Invalid_LOINC_code');
    fnCheckIfRecordReturned (false, SPM_OBX + '/ObservationIdentifier/AlternateIdentifier',
'LOINC_condition', '', 'OBX-3.4', 'WARNING', 'Condition_not_found_for_LOINC');
}
//-----OBX-3 (end)-----

if (ELRMessage.getField(SPM_OBX + '/ObservationResultStatus') != 'X') {
    //-----OBX-5-----
    //Handle OBX-5 variations based on the data type in OBX-2
    var OBX5DataType = ELRMessage.getField(SPM_OBX + '/ValueType');

    var requiredOBX5orOBX8 = false;
    if ((ELRMessage.getField(SPM_OBX + '/ObservationValue') == null || ELRMessage.getField
(SPM_OBX + '/ObservationValue') == '')
        && (ELRMessage.getField(SPM_OBX + '/AbnormalFlags[0]') == null || ELRMessage.getField
(SPM_OBX + '/AbnormalFlags[0]') == '')) {
        requiredOBX5orOBX8 = true;
    }
    else { requiredOBX5orOBX8 = false; }

    if ((OBX5DataType == 'CE') || (OBX5DataType == 'CWE')) {
        var OBX5CodingSys = ELRMessage.getField(SPM_OBX + '/ObservationValue/NameOfCodingSystem
');
        if ((OBX5CodingSys == 'SN') || (OBX5CodingSys == 'SNM')) {
            fnCheckIfRecordReturned (true, SPM_OBX + '/ObservationValue/Identifier', 'Snomed_code',
'', 'OBX-5.1', 'WARNING', 'Invalid_SNOMED_code');
            fnCheckIfRecordReturned (true, SPM_OBX + '/ObservationValue/Identifier', 'Snomed_con
dition', '', 'OBX-5.1', 'WARNING', 'Condition_not_found_for_SNOMED');
        }
        var OBX5AltCodingSys = next.getField(SPM_OBX + '/ObservationValue/NameOfAlternateCoding
System');
        if ((OBX5AltCodingSys == 'SN') || (OBX5AltCodingSys == 'SNM')) {
            fnCreateXMLSNOMEDEntry (false, SPM_OBX + '/ObservationValue/AlternateIdentifier',
'Snomed_code', '', 'OBX-5.4', 'WARNING', 'Invalid_SNOMED_code');
            fnCreateXMLSNOMEDEntry (false, SPM_OBX + '/ObservationValue/AlternateIdentifier',
'Snomed_condition', '', 'OBX-5.4', 'WARNING', 'Condition_not_found_for_SNOMED');

```

Javascript

```

    }
  }
  if (OBX5DataType == 'SN') {
    fnCheckIfRecordReturned (false, SPM_OBX + '/ObservationValue/Comparator', 'Code_value_general', 'OV_NUM_CMP', 'OBX-5.1', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, SPM_OBX + '/ObservationValue/SeparatorSuffix', 'Code_value_general', 'OV_NUM_SEP', 'OBX-5.3', 'INFO', 'Invalid_code');
  }
  //-----OBX-5 (end)-----

  //OBX-6 is required if OBX-2 is 'SN' or 'NM' and OBX-11 is not 'X'.
  fnCheckIfRecordReturned ((OBX5DataType == 'SN' || OBX5DataType == 'NM'), SPM_OBX + '/Units/Identifier', 'Code_value_general', 'UNIT_ISO', 'OBX-6.1', 'WARNING', 'Invalid_code');

  //ODS Comment: Presently NBS stores only one value
  fnCheckIfRecordReturned (requiredOBX5orOBX8, SPM_OBX + '/AbnormalFlags[0]/Identifier', 'Code_value_general', 'OBS_INTRP', 'OBX-8', 'INFO', 'Invalid_code');

  fnCheckIfRecordReturned (false, SPM_OBX + '/ObservationResultStatus', 'Code_value_general', 'ACT_OBJ_ST', 'OBX-11', 'INFO', 'Invalid_code');
  fnCheckIfRecordReturned (false, SPM_OBX + '/ProducersID/NameOfCodingSystem', 'Code_value_general', 'EI_AUTH', 'OBX-15.3', 'INFO', 'Invalid_code');

  //ODS Comment: Presently NBS stores only one value
  fnCheckIfRecordReturned (false, SPM_OBX + '/ObservationMethod[0]/Identifier', 'Code_value_general', 'OBS_METH', 'OBX-17.1', 'INFO', 'Invalid_code');

  fnCheckIfRecordReturned (false, SPM_OBX + '/PerformingOrganizationName/IDNumber_DEPRECATED', 'Code_value_general', 'EI_AUTH', 'OBX-23.3', 'INFO', 'Invalid_code');

  fnCheckIfRecordReturned (false, SPM_OBX + '/PerformingOrganizationAddress/StateOrProvince', 'State_code', 'STATE_CCD', 'OBX-24.4', 'INFO', 'Invalid_code');
  fnCheckIfRecordReturned (false, SPM_OBX + '/PerformingOrganizationAddress/Country', 'Code_value_general', 'PSL_CNTRY', 'OBX-24.6', 'INFO', 'Invalid_code');
  fnCheckIfRecordReturned (false, SPM_OBX + '/PerformingOrganizationAddress/CountyParishCode', 'V_State_county_code_value', 'COUNTY_CCD', 'OBX-24.9', 'INFO', 'Invalid_code');

  } // -----(END) if (ELRMessage.getField(SPM_OBX + '/ObservationResultStatus') != 'X')
  (END)

  } // -----(END) for (l=0; l < ELRMessage.getRepeatCount(SPM + '/OBX'); l++) (END)

  //----- OBX segments (end) -----

  } // -----(END) for (k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/SPECIMEN'); k++) (END)
  //----- SPECIMEN group (end) -----

  } // -----(END) for (j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT['+i+']/ORDER_OBSERVATION'); j++) (END)
  //--- ORDER_OBSERVATION group (end) ---

```

Javascript

```

} // -----(END)  for (i=0; i < ELRMessage.getRepeatCount('PATIENT_RESULT'); i++)  (END)
//----- PATIENT_RESULT group (end) -----

```

```
//
```

```
//===== Build Output XML file (end
```

```
)
```

```
//
```

```
next.text = dbInsertsXML;
```

Message Collector: Input Collation

Collection Order	[ECLIR - ELR 2.5.1 Message Validation/68580cb2-5571-4fb6-b51d-a4e7796446af,ECLIR - ELR 2.5.1 Message Validation/6fd292f8-d669-44e7-ba29-2cf12f762ade]
Match Property	ELR_MESSAGE_ID
Message Set Timeout	4
Timeout Units	Minutes
Timeout Destination	Process as Usual

Destinations

Name	Page
Output Condition: Empty Message	
Body does not contain 'errorRecord' ignoring case	
 ECLIR - ELR Sink.....Sink	14
Error Connector	Page
 db Add Field Descriptions.....Database Message Extraction	455



db ELR Validation Route Errors

Filter

Database LookUp

On ECLIR - ELR 2.5.1 Message Validation route

Notes

Purpose: This Database Lookup filter inserts an error record into the ELR_ERROR table in the ECLIR database in the event an unhandled error is caught by the *ECLIR - ELR 2.5.1 Message Validation Route Error Handler*.

Configuration Properties

Maximum Concurrency	1
---------------------	---

Configuration Properties		
Database		
Host		
Port		
Database Name		
Username		
Password		
Additional Connection Properties	Property Name	Property Value
Configuration File		
Message Type	Plain text	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	
Configuration File:		
<pre> <statement name="ELR_MESSAGE ID Does Not exist" ide:type="executeSql" test="Null(\$ELR_MESSAGE_ID)" ide:summary="The ELR_MESSAGE ID does not exist in the ELR_MESSAGE table. Create an entry in that table to create an ID." ide:allowCompoundStatements="true"> <sql>INSERT INTO ELR_MESSAGE (ORIGINAL_MESSAGE , ENVIRONMENT ,SUCCESS_FL ,INSERT_DATE) VALUES (\$messageContent , ENVIRONMENT ,0 ,SYSDATETIME())</sql> <statement ide:type="compoundSqlStatement"> <sql>SELECT \@ \@IDENTITY AS ELR_MESSAGE_ID</sql> <substituteColumn name="ELR_MESSAGE_ID">\$ELR_MESSAGE_ID</substituteColumn> </statement> <statement name="Insert into ELR_ERROR table" ide:type="executeSql" test="NotNull(@ELR_MESSAGE_ID)" ide:summary="Add error record to the ELR_ERROR table." ide:allowCompoundStatements="true"> <sql>INSERT INTO ELR_ERROR (ELR_MESSAGE_ID ,VALIDATION_TYPE ,ERROR_CATEGORY ,ERROR_TYPE ,PROCESSING_STEP ,ERROR_MSG ,INSERT_DATE) VALUES (</pre>		

Configuration File: [REDACTED]

```
@ELR_MESSAGE_ID
,$VALIDATION_TYPE
,$ERROR_CATEGORY
,$ERROR_TYPE
,$PROCESSING_STEP
,$ERROR_MSG
,$SYSDATETIME()
) </sql>
</statement>
</statement>
<statement name="Insert into ELR_ERROR table" ide:type="executeSql" test="NotNull($ELR_MESSAGE_ID)" ide:summary="Add error record to the ELR_ERROR table." ide:allowCompoundStatements="true">
  <sql>INSERT INTO ELR_ERROR (
    ELR_MESSAGE_ID
    ,VALIDATION_TYPE
    ,ERROR_CATEGORY
    ,ERROR_TYPE
    ,PROCESSING_STEP
    ,ERROR_MSG
    ,INSERT_DATE
  )
VALUES (
  $ELR_MESSAGE_ID
  , $VALIDATION_TYPE
  , $ERROR_CATEGORY
  , $ERROR_TYPE
  , $PROCESSING_STEP
  , $ERROR_MSG
  , $SYSDATETIME()
) </sql>
</statement>
</config>
```

Destinations

Name	Page
 no-op REH to Sink..... No-operation	456



NBS Definition Validation

HL7 Acknowledgement Generation

On ECLIR - ELR 2.5.1 Message Validation route

Filter

Notes

Purpose: This HL7 Acknowledgement Generation filter creates an ACK message type containing errors listing any violations of the ECLIR_ELROU_2.5.1_NBS_validation.s3d message definition. These errors are parsed out in the JavaScript filter *js Set Message Error properties* and written to an XML flat file for writing the errors to the ECLIR ELR_ERROR table.

Configuration Properties		
Message Definition	ECLIR_ELORU_2.5.1_NBS_validation.s3d	
Response Message Mapping		
Acknowledgement Mode	Validate, use Properties, and ACK	
NACK Mode	AR	
User Description		
On Message Validation Failure	Return Ack only to route	
On Successful Validation	Return Ack only to route	
Message Errors Property Name	MsgErrors	
HL7 Version	Same as input message	
Maintain Field Length	Ignore field lengths	
Fields to Copy	Source Field	Destination Field
	MSH.SendingApplication	MSH.ReceivingApplication
	MSH.SendingFacility	MSH.ReceivingFacility
	MSH.ReceivingApplication	MSH.SendingApplication
	MSH.ReceivingFacility	MSH.SendingFacility
	MSH.MessageControlId	MSA.MessageControlId
	MSH.ProcessingID	MSH.ProcessingID
Use Custom DateTime Format	false	

Destinations

Name	Page
 js Set Message Error properties.....JavaScript (v.1)	411



NIST Definition Validation

HL7 Acknowledgement Generation

On ECLIR - ELR 2.5.1 Message Validation route

Filter

Notes

Purpose: This HL7 Acknowledgement Generation filter creates an ACK message type containing errors listing any violations of the ECLIR_ELORU_2.5.1_NIST_validation.s3d message definition. These errors are parsed out in the JavaScript filter *js Set Message Error properties* and written to an XML flat file for writing the errors to the ECLIR ELR_ERROR table.

Configuration Properties	
Message Definition	ECLIR_ELORU_2.5.1_NIST_validation.s3d
Response Message Mapping	
Acknowledgement Mode	Validate, use Properties, and ACK
NACK Mode	AR
User Description	

Configuration Properties		
On Message Validation Failure	Return Ack only to route	
On Successful Validation	Return Ack only to route	
Message Errors Property Name	MsgErrors	
HL7 Version	Same as input message	
Maintain Field Length	Ignore field lengths	
Fields to Copy	Source Field	Destination Field
	MSH.SendingApplication	MSH.ReceivingApplication
	MSH.SendingFacility	MSH.ReceivingFacility
	MSH.ReceivingApplication	MSH.SendingApplication
	MSH.ReceivingFacility	MSH.SendingFacility
	MSH.MessageControlId	MSA.MessageControlId
	MSH.ProcessingID	MSH.ProcessingID
Use Custom DateTime Format	false	

Destinations

Name	Page
js Set Message Error properties.....JavaScript (v.1)	411